

Q1.1) Docker Image Tag

Create a Docker image named **ubuntu:latest** using a Dockerfile based on **ubuntu:22.04** with the **curl** package installed. Verify the installation by checking the curl version inside a container. Create additional tags **ubuntu:v1.0** and **ubuntu:stable**, and verify that all tags reference the same image. Remove the **ubuntu:stable** tag and confirm that **ubuntu:latest** and **ubuntu:v1.0** still exist and point to the original image.

Q1.2) Docker Image Build, Push, Pull and Container Management

Create a Docker image named **studentapp:v1.0** using a Dockerfile based on Ubuntu and install the **curl** package. Verify that the image works correctly by running a container and checking the curl version. Tag the image using your Docker Hub username and push it to your Docker Hub repository.

Remove the local image and verify that it is no longer available on the system. Pull the same image from Docker Hub and create a new container named **student_container** from the downloaded image. Access the container and install an additional package such as **apache2** or **vim**. Create a file containing your PRN number and name inside the container and verify its contents.

Commit the modified container as a new image version named **studentapp:v2.0**, push the updated image to Docker Hub, and verify that both image versions are available in your repository. Finally, remove the local images and containers, pull the latest version from Docker Hub, and verify that the changes made in version **v2.0** are preserved.

Q2) Docker Container Naming, Port Mapping and Package Installation

Create a Docker container named **web_container** using the Ubuntu image. Map container port **80** to host port **8080** and verify that the service is accessible from another machine using the host IP address. Access the running container, install the **apache2** package, start the service, create a custom web page displaying your PRN number, and verify that the page is accessible through the mapped port.

Q3) Docker Environment Variables (-e)

Create a container named **env_container** using the Nginx image and pass environment variables for **NAME**, **PRN**, and **COURSE** during container creation. Access the container and verify that the environment variables have been set correctly. Modify the web page to display the values stored in the environment variables and verify the output through a browser.

Q4) Docker Named Volume

Create a **named volume** called **student_data**. Launch a container named **vol_container** and attach the volume to a directory inside the container. Create a file containing your name and PRN number in the mounted directory. Remove the container and create a new container using the same volume. Verify that the previously created file is still available.

Q5) Docker Anonymous Volume

Create a container using an **anonymous volume** mounted to a data directory. Create a file containing your PRN number and verify its existence. Remove the container and create a new container from the same image. Verify that the previously stored data is not automatically available and explain the behavior of anonymous volumes.

Q6) Docker Bind Mount

Create a directory on the host machine named **docker_bind_data** and place a file containing your name and PRN number inside it. Launch a container named **bind_container** and bind mount the host directory into the container. Verify that the file is accessible from inside the container. Modify the file from within the container and verify that the changes are reflected on the host machine. Finally, modify the file on the host and verify that the updated content is visible inside the container.

Q1.1) Docker Image Tag

Create a Docker image named **ubuntu:latest** using a Dockerfile based on **ubuntu:22.04** with the **curl** package installed. Verify the installation by checking the curl version inside a container. Create additional tags **ubuntu:v1.0** and **ubuntu:stable**, and verify that all tags reference the same image. Remove the **ubuntu:stable** tag and confirm that **ubuntu:latest** and **ubuntu:v1.0** still exist and point to the original image.

Step 1: Create a Working Directory

We create a separate directory to store the Dockerfile and related files for building the custom Ubuntu image.

```
PS C:\Users\Lenovo> cd "D:\learn\itim\itim\docker\newLab"
PS D:\learn\itim\itim\docker\newLab> mkdir q1_imgTag

Directory: D:\learn\itim\itim\docker\newLab

Mode                LastWriteTime         Length Name
----                -
d-----          07-06-2026         15:09         q1_imgTag

PS D:\learn\itim\itim\docker\newLab> cd
PS D:\learn\itim\itim\docker\newLab> cd .\q1_imgTag\
PS D:\learn\itim\itim\docker\newLab\q1_imgTag> |
```

Step 2: Create the Dockerfile

We create a Dockerfile that uses Ubuntu 22.04 as the base image and installs the curl package during image creation.

Command

code Dockerfile

```
PS D:\learn\itim\itim\docker\newLab\q1_imgTag> code Dockerfile
PS D:\learn\itim\itim\docker\newLab\q1_imgTag> cat .\Dockerfile
FROM ubuntu:22.04
RUN apt update && apt install -y curl
CMD ["/bin/bash"]
```

Dockerfile Content

FROM ubuntu:22.04

RUN apt-get update && apt-get install -y curl

CMD ["/bin/bash"]

Explanation of attributes

- **FROM ubuntu:22.04** → Specifies Ubuntu 22.04 as the base image.
- **RUN** → Executes commands while building the image.
- **apt-get update** → Updates package information.
- **apt-get install -y curl** → Installs curl automatically.
- **CMD** → Default command executed when a container starts.

Step 3: Build the Docker Image

We build the Docker image and assign the tag **ubuntu:latest**.

Command

`docker build -t ubuntu:latest .`

```
PS D:\learn\itim\itim\docker\newLab\q1_imgTag> docker build -t ubuntu:latest .
[+] Building 24.0s (6/6) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 112B                             0.0s
=> [internal] load metadata for docker.io/library/ubuntu:22.04  4.9s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                    0.0s
=> [1/2] FROM docker.io/library/ubuntu:22.04@sha256:4f838adc7181d9039ac795a7d0aba05a9bd9ecd480d2944831  5.5s
=> => resolve docker.io/library/ubuntu:22.04@sha256:4f838adc7181d9039ac795a7d0aba05a9bd9ecd480d2944831  0.0s
=> => sha256:4f838adc7181d9039ac795a7d0aba05a9bd9ecd480d294483169c5def983b64d 6.69kB / 6.69kB 0.0s
=> => sha256:ce941a2a18bbb922e434d6d6d2b31e571a5c3826eaf6ada0a41dcc905bd2d906 424B / 424B 0.0s
=> => sha256:86f1a8d7b38e7a014c249cf2ca573c8ff7ce3cca128c5c06dcee758813726f90 2.05kB / 2.05kB 0.0s
=> => sha256:40d16f30db405106ef8074779bdf41f012465c2a785bbeaa2eab9f2081099b47 29.74MB / 29.74MB 2.3s
=> => extracting sha256:40d16f30db405106ef8074779bdf41f012465c2a785bbeaa2eab9f2081099b47 2.9s
=> [2/2] RUN apt update && apt install -y curl                  13.8s
=> exporting to image                                           0.5s
=> => exporting layers                                           0.5s
=> => writing image sha256:5d80a4536ff51f2024c6554a01dce536005366d8293e7fed9237d3baa0b51ef0 0.0s
=> => naming to docker.io/library/ubuntu:latest                 0.0s

What's next:
View a summary of image vulnerabilities and recommendations → docker scout quickview
```

Explanation of attributes

- **docker build** → Creates a Docker image.
- **-t** → Assigns a name and tag to the image.
- **ubuntu:latest** → Image name and tag.
- **.** → Uses the current directory as the build context.

Step 4: Verify Image Creation

We confirm that the image has been successfully created.

Command

`docker images`

```
PS D:\learn\itim\itim\docker\newLab\q1_imgTag> docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
ubuntu	latest	5d80a4536ff5	18 seconds ago	162MB
nginx	<none>	7aaca76c508f	2 weeks ago	161MB
ubuntu	<none>	30ba44506a6d	6 weeks ago	100MB
mongo	8.0.0-rc11-jammy	e9bfc4212c57	23 months ago	867MB
server	latest	14bac6b71a17	2 years ago	912MB
postgres	16.1-alpine3.18	b467784150b4	2 years ago	239MB

Explanation of attributes

- `docker images` → Lists all available Docker images.

Step 5: Run a Container and Verify curl Installation

We start a container from the image and verify that curl was installed correctly.

Command

`docker run -it ubuntu:latest bash`

```
PS D:\learn\itim\itim\docker\newLab\q1_imgTag> docker run -it ubuntu:latest
root@bcc329cddb62:/# curl --version
curl 7.81.0 (x86_64-pc-linux-gnu) libcurl/7.81.0 OpenSSL/3.0.2 zlib/1.2.11 brotli/1.0.9 zstd/1.4.8 libidn2/2.3
.2 libpsl/0.21.0 (+libidn2/2.3.2) libssh/0.9.6/openssl/zlib nghttp2/1.43.0 librtmp/2.3 OpenLDAP/2.5.20
Release-Date: 2022-01-05
Protocols: dict file ftps gopher gophers http https imap imaps ldap ldaps mqtt pop3 pop3s rtmp rtsp scp sf
tp smb smbs smtp smtps telnet tftp
Features: alt-svc AsynchDNS brotli GSS-API HSTS HTTP2 HTTPS-proxy IDN IPv6 Kerberos Largefile libz NTLM NTLM_W
B PSL SPNEGO SSL TLS-SRP UnixSockets zstd
```

Explanation of attributes

- `docker run` → Creates and starts a container.
- `-i` → Keeps STDIN open.
- `-t` → Allocates a terminal.
- `ubuntu:latest` → Image used to create the container.
- `bash` → Starts a Bash shell inside the container.

Verify curl

curl --version

```
PS D:\learn\itim\itim\docker\newLab\q1_imgTag> docker run -it ubuntu:latest
root@bcc329cddb62:/# curl --version
curl 7.81.0 (x86_64-pc-linux-gnu) libcurl/7.81.0 OpenSSL/3.0.2 zlib/1.2.11 brotli/1.0.9 zstd/1.4.8 libidn2/2.3
.2 libpsl/0.21.0 (+libidn2/2.3.2) libssh/0.9.6/openssl/zlib nghttp2/1.43.0 librtmp/2.3 OpenLDAP/2.5.20
Release-Date: 2022-01-05
Protocols: dict file ftp ftps gopher gophers http https imap imaps ldap ldaps mqtt pop3 pop3s rtmp rtsp scp sf
tp smb smbs smtp smtps telnet tftp
Features: alt-svc AsynchDNS brotli GSS-API HSTS HTTP2 HTTPS-proxy IDN IPv6 Kerberos Largefile libz NTLM NTLM_W
B PSL SPNEGO SSL TLS-SRP UnixSockets zstd
```

Explanation of attributes

- `curl --version` → Displays the installed curl version.

Step 6: Create the Tag ubuntu:v1.0

We create a second tag that points to the same image, representing version 1.0.

Command

docker tag ubuntu:latest ubuntu:v1.0

```
PS D:\learn\itim\itim\docker\newLab\q1_imgTag> docker tag ubuntu:latest ubuntu:v1.0
PS D:\learn\itim\itim\docker\newLab\q1_imgTag> docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
ubuntu	latest	5d80a4536ff5	5 minutes ago	162MB
ubuntu	v1.0	5d80a4536ff5	5 minutes ago	162MB
nginx	<none>	7aaca76c508f	2 weeks ago	161MB
ubuntu	<none>	30ba44506a6d	6 weeks ago	100MB
mongo	8.0.0-rc11-jammy	e9bfc4212c57	23 months ago	867MB
server	latest	14bac6b71a17	2 years ago	912MB
postgres	16.1-alpine3.18	b467784150b4	2 years ago	239MB

Explanation of attributes

- `docker tag` → Creates a new tag for an existing image.
- First image name → Source image.
- Second image name → New tag.

Step 7: Create the Tag ubuntu:stable

We create another tag representing the stable release of the image.

Command

```
docker tag ubuntu:latest ubuntu:stable
```

```
PS D:\learn\itim\itim\docker\newLab\q1_imgTag> docker tag ubuntu:latest ubuntu:stable
PS D:\learn\itim\itim\docker\newLab\q1_imgTag> docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
ubuntu	latest	5d80a4536ff5	6 minutes ago	162MB
ubuntu	stable	5d80a4536ff5	6 minutes ago	162MB
ubuntu	v1.0	5d80a4536ff5	6 minutes ago	162MB

Explanation of attributes

- `ubuntu:latest` → Existing image.
- `ubuntu:stable` → New stable tag.

Step 8: Verify All Tags

We verify that all three tags exist and reference the same image ID.

Command

```
docker images
```

```
PS D:\learn\itim\itim\docker\newLab\q1_imgTag> docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
ubuntu	latest	5d80a4536ff5	6 minutes ago	162MB
ubuntu	stable	5d80a4536ff5	6 minutes ago	162MB
ubuntu	v1.0	5d80a4536ff5	6 minutes ago	162MB

Explanation of attributes

- Check that:
 - `ubuntu:latest`
 - `ubuntu:v1.0`
 - `ubuntu:stable`
- All have the same IMAGE ID.

Step 9: Remove the ubuntu:stable Tag

We remove only the stable tag while keeping the underlying image intact.

Command

```
docker rmi ubuntu:stable
```

```
PS D:\learn\itim\itim\docker\newLab\q1_imgTag> docker rmi ubuntu:stable
Untagged: ubuntu:stable
PS D:\learn\itim\itim\docker\newLab\q1_imgTag> docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
ubuntu	latest	5d80a4536ff5	8 minutes ago	162MB
ubuntu	v1.0	5d80a4536ff5	8 minutes ago	162MB

Explanation of attributes

- `docker rmi` → Removes an image tag.
- `ubuntu:stable` → Tag to remove.

Step 10: Verify Remaining Tags

We ensure that removing one tag does not delete the image because other tags still reference it.

Command

```
docker images
```

```
PS D:\learn\itim\itim\docker\newLab\q1_imgTag> docker rmi ubuntu:stable
Untagged: ubuntu:stable
PS D:\learn\itim\itim\docker\newLab\q1_imgTag> docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
ubuntu	latest	5d80a4536ff5	8 minutes ago	162MB
ubuntu	v1.0	5d80a4536ff5	8 minutes ago	162MB

Explanation of attributes

- Verify:
 - `ubuntu:latest` exists.
 - `ubuntu:v1.0` exists.
 - Both point to the same IMAGE ID.

Expected Outcome

- Docker image **ubuntu:latest** is created successfully.
- curl package is installed and verified.
- Additional tags **ubuntu:v1.0** and **ubuntu:stable** are created.
- All tags reference the same image ID.
- The **ubuntu:stable** tag is removed successfully.
- **ubuntu:latest** and **ubuntu:v1.0** continue to exist and reference the original image.

Memory Sheet (Command Flow)

```
mkdir dockerQ3
```

```
cd dockerQ3
```

```
code Dockerfile
```

```
docker build -t ubuntu:latest .
```

```
docker images
```

```
docker run -it ubuntu:latest bash
```

```
curl --version
```

```
exit
```

```
docker tag ubuntu:latest ubuntu:v1.0
```

```
docker tag ubuntu:latest ubuntu:stable
```

```
docker images
```

```
docker rmi ubuntu:stable
```

```
docker images
```

Q1.2) Docker Image Build, Push, Pull and Container Management

Create a Docker image named **studentapp:v1.0** using a Dockerfile based on Ubuntu and install the **curl** package. Verify that the image works correctly by running a container and checking the curl version. Tag the image using your Docker Hub username and push it to your Docker Hub repository.

Remove the local image and verify that it is no longer available on the system. Pull the same image from Docker Hub and create a new container named **student_container** from the downloaded image. Access the container and install an additional package such as **apache2** or **vim**. Create a file containing your PRN number and name inside the container and verify its contents.

Commit the modified container as a new image version named **studentapp:v2.0**, push the updated image to Docker Hub, and verify that both image versions are available in your repository. Finally, remove the local images and containers, pull the latest version from Docker Hub, and verify that the changes made in version **v2.0** are preserved.

—

Step 1: Create a Working Directory

We create a dedicated directory to store the Dockerfile and related files required to build the custom Docker image.

Command

```
mkdir studentapp
```

```
cd studentapp
```

Step 2: Create the Dockerfile

We create a Dockerfile based on Ubuntu and install the curl package during image creation.

Command code Dockerfile

Dockerfile Content

FROM ubuntu:22.04

RUN apt-get update && apt-get install -y curl

CMD ["/bin/bash"]

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> cat .\Dockerfile
FROM ubuntu:22.04
RUN apt-get update && apt-get install -y curl
CMD ["/bin/bash"]
```

Explanation of attributes

- **FROM** → Specifies the base image.
- **RUN** → Executes commands while building.
- **apt-get update** → Updates package information.
- **apt-get install -y curl** → Installs curl.
- **CMD** → Default command executed when container starts.

Step 3: Build the Docker Image

We build the image and assign the tag studentapp:v1.0.

Command

docker build -t studentapp:v1.0 .

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker build -t studentapp:v1.0 .
[+] Building 16.3s (6/6) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 120B                             0.0s
=> [internal] load metadata for docker.io/library/ubuntu:22.04  2.4s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                    0.0s
=> CACHED [1/2] FROM docker.io/library/ubuntu:22.04@sha256:4f838adc7181d9039ac795a7d0aba05a9bd9ecd480d 0.0s
=> [2/2] RUN apt-get update && apt-get install -y curl          13.7s
=> exporting to image                                           0.9s
=> => exporting layers                                           0.9s
=> => writing image sha256:3c7be722a52ff0203646d59de60aa102f932e5bf977e1c62f6985841c9f48e7e 0.0s
=> => naming to docker.io/library/studentapp:v1.0              0.0s

What's next:
View a summary of image vulnerabilities and recommendations → docker scout quickview
```

Explanation of attributes

- **docker build** → Creates Docker image.
- **-t** → Assigns image name and tag.

- `studentapp:v1.0` → Image name and version.
- `.` → Current directory as build context.

Step 4: Verify the Image

We verify that the image was successfully created.

Command

`docker images`

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker images
REPOSITORY      TAG              IMAGE ID         CREATED          SIZE
studentapp      v1.0            3c7be722a52f    40 seconds ago  162MB
server          latest          14bac6b71a17    2 years ago     912MB
postgres        16.1-alpine3.18 b467784150b4    2 years ago     239MB
```

Explanation of attributes

- `docker images` → Lists all available images.

Step 5: Run a Container and Verify curl

We launch a container from the image and confirm that curl is installed.

Command

`docker run -it studentapp:v1.0 bash`

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker run -it studentapp:v1.0 bash
root@d412ad2796a3:/# ls
bin  dev  home  lib32  libx32  mnt  proc  run  srv  tmp  var
boot  etc  lib  lib64  media  opt  root  sbin  sys  usr
root@d412ad2796a3:/# curl -V
curl 7.81.0 (x86_64-pc-linux-gnu) libcurl/7.81.0 OpenSSL/3.0.2 zlib/1.2.11 brotli/1.0.9 zstd/1.4.8 libidn2/2.3
.2 libpsl/0.21.0 (+libidn2/2.3.2) libssh/0.9.6/openssl/zlib nghttp2/1.43.0 librtmp/2.3 OpenLDAP/2.5.20
Release-Date: 2022-01-05
Protocols: dict file ftp ftps gopher gophers http https imap imaps ldap ldaps mqtt pop3 pop3s rtmp rtsp scp sf
tp smb smbs smtp smtps telnet tftp
Features: alt-svc AsynchDNS brotli GSS-API HSTS HTTP2 HTTPS-proxy IDN IPv6 Kerberos Largefile libz NTLM NTLM_W
B PSL SPNEGO SSL TLS-SRP UnixSockets zstd
root@d412ad2796a3:/# exit
exit
```

Explanation of attributes

- `docker run` → Creates and starts a container.
- `-it` → Interactive terminal mode.
- `studentapp:v1.0` → Source image.
- `bash` → Starts shell.

Verification Command

`curl --version`

Explanation of attributes

- `curl --version` → Displays installed curl version.

Exit Container

Exit

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker run -it studentapp:v1.0 bash
root@d412ad2796a3:/# ls
bin  dev  home  lib32  libx32  mnt  proc  run  srv  tmp  var
boot  etc  lib  lib64  media  opt  root  sbin  sys  usr
root@d412ad2796a3:/# curl -V
curl 7.81.0 (x86_64-pc-linux-gnu) libcurl/7.81.0 OpenSSL/3.0.2 zlib/1.2.11 brotli/1.0.9 zstd/1.4.8 libidn2/2.3
.2 libpsl/0.21.0 (+libidn2/2.3.2) libssh/0.9.6/openssl/zlib nghttp2/1.43.0 librtmp/2.3 OpenLDAP/2.5.20
Release-Date: 2022-01-05
Protocols: dict file ftp ftps gopher gophers http https imap imaps ldap ldaps mqtt pop3 pop3s rtmp rtsp scp sf
tp smb smbs smtp smtps telnet tftp
Features: alt-svc AsynchDNS brotli GSS-API HSTS HTTP2 HTTPS-proxy IDN IPv6 Kerberos Largefile libz NTLM NTLM_W
B PSL SPNEGO SSL TLS-SRP UnixSockets zstd
root@d412ad2796a3:/# exit
exit
```

Step 6: Tag Image for Docker Hub

We prepare the image for uploading to Docker Hub.

Command

`docker tag studentapp:v1.0 <DOCKERHUB_USERNAME>/studentapp:v1.0`

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker tag studentapp:v1.0 chief7/studentapp:v1
.0
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker images
REPOSITORY          TAG          IMAGE ID          CREATED          SIZE
chief7/studentapp   v1.0         3c7be722a52f     5 minutes ago   162MB
studentapp          v1.0         3c7be722a52f     5 minutes ago   162MB
server              latest       14bac6b71a17     2 years ago     912MB
postgres            16.1-alpine3.18 b467784150b4     2 years ago     239MB
```

Explanation of attributes

- `docker tag` → Creates additional image tag.
- `<DOCKERHUB_USERNAME>` → Your Docker Hub username.
- `studentapp:v1.0` → Image version.

Step 7: Login to Docker Hub

We authenticate with Docker Hub before pushing images.

Command

docker login

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker login

USING WEB-BASED LOGIN

Info → To sign in with credentials on the command line, use 'docker login -u <username>'

Your one-time device confirmation code is: WXQZ-QBKK
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate

Waiting for authentication in the browser...
Login Succeeded
```

Explanation of attributes

- Prompts for Docker Hub credentials.

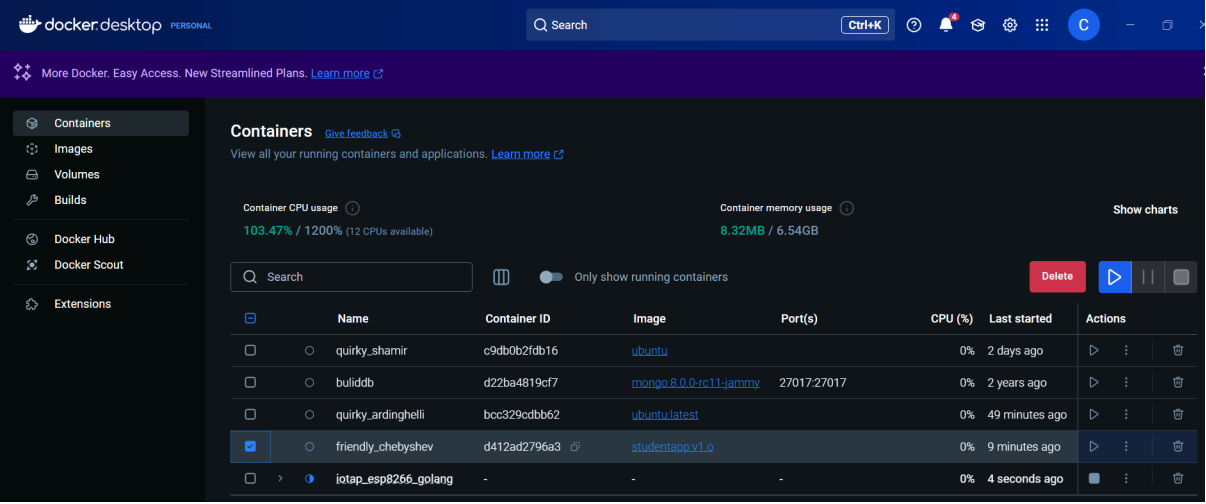
Step 8: Push Image to Docker Hub

We upload the image to Docker Hub for remote storage.

Command

docker push <DOCKERHUB_USERNAME>/studentapp:v1.0

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker push chief7/studentapp:v1.0
The push refers to repository [docker.io/chief7/studentapp]
604e0e31610b: Pushed
8bba68e76219: Mounted from library/ubuntu
v1.0: digest: sha256:449977b4e3357b4ebe1b3a86003c15c8051962943883d91a43a86b2695366c9a size: 741
```



The screenshot shows the Docker Desktop interface. On the left is a sidebar with navigation options: Containers, Images, Volumes, Builds, Docker Hub, Docker Scout, and Extensions. The main area is titled 'Containers' and displays a table of running containers. Above the table, it shows 'Container CPU usage' at 103.47% / 1200% (12 CPUs available) and 'Container memory usage' at 8.32MB / 6.54GB. There is a search bar and a toggle for 'Only show running containers'. The table lists five containers: 'quirky_shamir' (ubuntu), 'bulddb' (mongo:8.0.0-rc11-jammy), 'quirky_ardinghelli' (ubuntu:latest), 'friendly_chebyshev' (studentapp:v1.0), and 'iotap_esp8266_golang'.

Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
quirky_shamir	c9db062fdb16	ubuntu		0%	2 days ago	[Play] [More] [Delete]
bulddb	d22ba4819cf7	mongo:8.0.0-rc11-jammy	27017:27017	0%	2 years ago	[Play] [More] [Delete]
quirky_ardinghelli	bcc329cddb62	ubuntu:latest		0%	49 minutes ago	[Play] [More] [Delete]
friendly_chebyshev	d412ad2796a3	studentapp:v1.0		0%	9 minutes ago	[Play] [More] [Delete]
iotap_esp8266_golang	-	-	-	0%	4 seconds ago	[Play] [More] [Delete]

Explanation of attributes

- `docker push` → Uploads image to Docker Hub.
- Repository name identifies image location.

Step 9: Remove Local Image

We remove the local image to verify that it can be downloaded from Docker Hub.

Command

```
docker rmi <DOCKERHUB_USERNAME>/studentapp:v1.0
```

```
docker rmi studentapp:v1.0
```

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker rmi chief7/studentapp:v1.0
Untagged: chief7/studentapp:v1.0
Untagged: chief7/studentapp@sha256:449977b4e3357b4ebe1b3a86003c15c8051962943883d91a43a86b2695366c9a
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker rmi studentapp:v1.0
Error response from daemon: conflict: unable to remove repository reference "studentapp:v1.0" (must force) - c
ontainer d412ad2796a3 is using its referenced image 3c7be722a52f
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker rmi -f studentapp:v1.0
Untagged: studentapp:v1.0
Deleted: sha256:3c7be722a52ff0203646d59de60aa102f932e5bf977e1c62f6985841c9f48e7e
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker images
REPOSITORY      TAG          IMAGE ID      CREATED      SIZE
server           latest       14bac6b71a17  2 years ago  912MB
postgres         16.1-alpine3.18  b467784150b4  2 years ago  239MB
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> |
```

- `docker rmi` → Removes local image.

Step 10: Verify Image Removal

We confirm that the image is no longer available locally.

```
docker images
```

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker images
REPOSITORY      TAG          IMAGE ID      CREATED      SIZE
server           latest       14bac6b71a17  2 years ago  912MB
postgres         16.1-alpine3.18  b467784150b4  2 years ago  239MB
```

Explanation of attributes

- Ensures image has been removed.

Step 11: Pull Image from Docker Hub

We download the previously uploaded image.

Command

```
docker pull <DOCKERHUB_USERNAME>/studentapp:v1.0
```

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker pull chief7/studentapp:v1.0
v1.0: Pulling from chief7/studentapp
40d16f30db40: Already exists
d709791230c1: Already exists
Digest: sha256:449977b4e3357b4ebe1b3a86003c15c8051962943883d91a43a86b2695366c9a
Status: Downloaded newer image for chief7/studentapp:v1.0
docker.io/chief7/studentapp:v1.0

What's next:
  View a summary of image vulnerabilities and recommendations → docker scout quickview chief7/studentapp:v1.0
```

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
chief7/studentapp    v1.0               3c7be722a52f       9 minutes ago      162MB
server               latest             14bac6b71a17       2 years ago        912MB
postgres             16.1-alpine3.18    b467784150b4       2 years ago        239MB
```

Explanation of attributes

- **docker pull** → Downloads image from repository.

Step 12: Create a New Container

We create a container named `student_container` from the downloaded image.

Command

```
docker run -it --name student_container
<DOCKERHUB_USERNAME>/studentapp:v1.0 bash
```

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker run -it --name student_container chief7/studentapp:v1.0 bash
root@b3f56ca3b6ce:/# curl -v
curl: no URL specified!
curl: try 'curl --help' or 'curl --manual' for more information
root@b3f56ca3b6ce:/# curl -V
curl 7.81.0 (x86_64-pc-linux-gnu) libcurl/7.81.0 OpenSSL/3.0.2 zlib/1.2.11 brotli/1.0.9 zstd/1.4.8 libidn2/2.3.2 libpsl/0.21.0 (+libidn2/2.3.2) libssh/0.9.6/openssl/zlib nghttp2/1.43.0 librtmp/2.3 OpenLDAP/2.5.20
Release-Date: 2022-01-05
Protocols: dict file ftp ftps gopher gophers http https imap imaps ldap ldaps mqtt pop3 pop3s rtmp rtsp scp sftp smb smbs smtp smtps telnet tftp
Features: alt-svc AsynchDNS brotli GSS-API HSTS HTTP2 HTTPS-proxy IDN IPv6 Kerberos Largefile libz NTLM NTLM_WB PSL SPNEGO SSL TLS-SRP
UnixSockets zstd
root@b3f56ca3b6ce:/# |
```

Explanation of attributes

- **--name** → Assigns container name.
- **student_container** → Container identifier.

Step 13: Install Additional Package

We modify the running container by installing an additional package.

Command

`apt-get update`

`apt-get install -y vim`

```
root@b3f56ca3b6ce:/# apt-get install -y vim
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libexpat1 libgpm2 libmpdec3 libpython3.10 libpython3.10-minimal libpython3.10-stdlib libreadline8 libsodium23
  media-types readline-common vim-common vim-runtime xxd
Suggested packages:
  gpm readline-doc ctags vim-doc vim-scripts
The following NEW packages will be installed:
  libexpat1 libgpm2 libmpdec3 libpython3.10 libpython3.10-minimal libpython3.10-stdlib libreadline8 libsodium23
  media-types readline-common vim vim-common vim-runtime xxd
0 upgraded, 15 newly installed, 0 to remove and 3 not upgraded.
Need to get 14.5 MB of archives.
After this operation, 61.2 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libexpat1 amd64 2.4.7-1ubuntu0.7 [92.1 kB]
```

Explanation of attributes

- Installs Vim editor inside the container.

Step 14: Create Student Information File

We create a file containing PRN and student name inside the container.

Command

`echo "PRN: <PRN_NUMBER>" > student.txt`

`echo "Name: <FULL_NAME>" >> student.txt`

```
root@b3f56ca3b6ce:/# echo "PRN: 260240123060" > student.txt
root@b3f56ca3b6ce:/# echo "PRN: John j J jr" >> student.txt
root@b3f56ca3b6ce:/# cat student.txt
PRN: 260240123060
PRN: John j J jr
```

Explanation of attributes

- `>` → Creates file.
- `>>` → Appends content.

Step 15: Verify File Contents

We confirm that the file contains the required information.

Command

cat student.txt

```
root@b3f56ca3b6ce:/# echo "PRN: 260240123060" > student.txt
root@b3f56ca3b6ce:/# echo "PRN: John j J jr" >> student.txt
root@b3f56ca3b6ce:/# cat student.txt
PRN: 260240123060
PRN: John j J jr
```

Explanation of attributes

- **cat** → Displays file content.

Step 16: Exit the Container

We leave the container while preserving its state.

Command

exit

```
root@b3f56ca3b6ce:/# exit
exit
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> |
```

Step 17: Commit Container as Version 2.0

We save the modified container as a new Docker image.

Command

```
docker commit student_container
<DOCKERHUB_USERNAME>/studentapp:v2.0
```

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker commit student_container chief7/studentapp:v2.0
sha256:2dd79aa720323f709b12a1701b2337438c14550e93627a98dc7fc2238b167451
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
chief7/studentapp    v2.o               2dd79aa72032       46 seconds ago     222MB
chief7/studentapp    v1.o               3c7be722a52f       19 minutes ago     162MB
server               latest             14bac6b71a17       2 years ago        912MB
postgres             16.1-alpine3.18    b467784150b4       2 years ago        239MB
```

Explanation of attributes

- `commit` → Creates image from container.
- `student_container` → Source container.
- `studentapp:v2.0` → New image version.

Step 18: Push Version 2.0 to Docker Hub

We upload the updated image to Docker Hub.

Command

```
docker push <DOCKERHUB_USERNAME>/studentapp:v2.0
```

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker push chief7/studentapp:v2.0
The push refers to repository [docker.io/chief7/studentapp]
6d1c1f96f3d7: Pushed
604e0e31610b: Layer already exists
8bba68e76219: Layer already exists
v2.0: digest: sha256:288486141704715a3ed00eb089390e237ac0a64b792a03c52677ac8120f5be7f size: 953
```

Container CPU usage ⓘ
No containers are running.

Container memory usage ⓘ
No containers are running.

Show charts

☐ Only show running containers Delete ▶ || □

	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	quirky_shamir	c9db0b2fdb16	ubuntu		N/A	2 days ago	
☐	bulddb	d22ba4819cf7	mongo:8.0.0-rc11-jammy	27017:27017	N/A	2 years ago	
☐	quirky_ardinghelli	bcc329cd6b62	ubuntu:latest		N/A	1 hour ago	
☐	friendly_chebyshev	d412ad2796a3	studentapp:v1.0		N/A	21 minutes ago	
☒	student_container	b3f56ca3b6ce	chief7/studentapp:v1.0		N/A	10 minutes ago	
☐	> iotap_esp8266_golang	-	-	-	N/A	11 seconds ago	

Explanation of attributes

- Uploads new version.

Step 19: Verify Both Versions

We ensure both image versions are available.

Command

docker images

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker commit student_container chief7/studentapp:v2.0
sha256:2dd79aa720323f709b12a1701b2337438c14550e93627a98dc7fc2238b167451
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
chief7/studentapp    v2.0               2dd79aa72032       46 seconds ago     222MB
chief7/studentapp    v1.0               3c7be722a52f       19 minutes ago     162MB
server               latest             14bac6b71a17       2 years ago        912MB
postgres             16.1-alpine3.18    b467784150b4       2 years ago        239MB
```

Explanation of attributes

- Verify:
 - studentapp:v1.0
 - studentapp:v2.0

Step 20: Remove Local Images and Container

We clean up the local system before verification.

Command

docker rm student_container

docker rmi <DOCKERHUB_USERNAME>/studentapp:v1.0

docker rmi <DOCKERHUB_USERNAME>/studentapp:v2.0

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker rmi -f chief7/studentapp:v2.0
Untagged: chief7/studentapp:v2.0
Untagged: chief7/studentapp@sha256:288486141704715a3ed00eb089390e237ac0a64b792a03c52677ac8120f5be7f
Deleted: sha256:2dd79aa720323f709b12a1701b2337438c14550e93627a98dc7fc2238b167451
Deleted: sha256:fcc1c8764bad5571a82a042708b62be9788e9f5bc0d10ae77e9fe62181b234ee
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker rmi -f chief7/studentapp:v1.0
Untagged: chief7/studentapp:v1.0
Untagged: chief7/studentapp@sha256:449977b4e3357b4ebe1b3a86003c15c8051962943883d91a43a86b2695366c9a
Deleted: sha256:3c7be722a52ff0203646d59de60aa102f932e5bf977e1c62f6985841c9f48e7e
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
server               latest             14bac6b71a17       2 years ago        912MB
postgres             16.1-alpine3.18    b467784150b4       2 years ago        239MB
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> |
```

Explanation of attributes

- **rm** → Removes container.
- **rmi** → Removes image.
-

Step 21: Pull Version 2.0 from Docker Hub

We download the latest image and verify persistence.

Command

```
docker pull <DOCKERHUB_USERNAME>/studentapp:v2.0
```

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker pull chief7/studentapp:v2.0
v2.0: Pulling from chief7/studentapp
40d16f30db40: Already exists
d709791230c1: Already exists
5a8272c463f4: Pull complete
Digest: sha256:288486141704715a3ed00eb089390e237ac0a64b792a03c52677ac8120f5be7f
Status: Downloaded newer image for chief7/studentapp:v2.0
docker.io/chief7/studentapp:v2.0
```

Explanation of attributes

- Retrieves latest version.

Step 22: Verify Preserved Changes

We confirm that the installed package and student file exist in version 2.0.

Command

```
docker run -it <DOCKERHUB_USERNAME>/studentapp:v2.0 bash
```

```
cat student.txt
```

```
vim --version
```

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker run -it chief7/studentapp:v2.0 bash
root@2c9cf00889af:/# cat student.txt
PRN: 260240123060
PRN: John j J jr
root@2c9cf00889af:/# vim --version
VIM - Vi IMproved 8.2 (2019 Dec 12, compiled May 20 2026 21:28:11)
Included patches: 1-16, 647, 17-579, 1969, 580-647, 678, 648-1848, 4975, 5016, 5023, 5072, 2068, 1849-1854, 18
8, 1858-1859, 1873, 1860-1969, 1992, 1970-1992, 2010, 1993-2068, 2106, 2069-2106, 2108, 2107-2109, 2109-3995,
4899, 4901, 4919, 213, 1840, 1846-1847, 2110-2112, 2121
Modified by team+vim@tracker.debian.org
Compiled by team+vim@tracker.debian.org
```

Explanation of attributes

- `cat student.txt` → Verifies stored file.
- `vim --version` → Confirms installed package.

Memory Sheet (Command Flow)

mkdir studentapp

cd studentapp

vim Dockerfile

docker build -t studentapp:v1.0 .

docker images

docker run -it studentapp:v1.0 bash

curl --version

exit

docker tag studentapp:v1.0 <DOCKERHUB_USERNAME>/studentapp:v1.0

docker login

docker push <DOCKERHUB_USERNAME>/studentapp:v1.0

docker rmi <DOCKERHUB_USERNAME>/studentapp:v1.0

docker rmi studentapp:v1.0

docker pull <DOCKERHUB_USERNAME>/studentapp:v1.0

docker run -it --name student_container
<DOCKERHUB_USERNAME>/studentapp:v1.0 bash

apt-get update

apt-get install -y vim

echo "PRN: <PRN_NUMBER>" > student.txt

echo "Name: <FULL_NAME>" >> student.txt

cat student.txt

exit

```
docker                                commit                                student_container  
<DOCKERHUB_USERNAME>/studentapp:v2.0
```

```
docker push <DOCKERHUB_USERNAME>/studentapp:v2.0
```

```
docker images
```

```
docker rm student_container
```

```
docker rmi <DOCKERHUB_USERNAME>/studentapp:v1.0
```

```
docker rmi <DOCKERHUB_USERNAME>/studentapp:v2.0
```

```
docker pull <DOCKERHUB_USERNAME>/studentapp:v2.0
```

```
docker run -it <DOCKERHUB_USERNAME>/studentapp:v2.0 bash
```

```
cat student.txt
```

```
vim --version
```

Q2) Docker Container Naming, Port Mapping and Package Installation

Create a Docker container named **web_container** using the Ubuntu image. Map container port **80** to host port **8080** and verify that the service is accessible from another machine using the host IP address. Access the running container, install the **apache2** package, start the service, create a custom web page displaying your PRN number, and verify that the page is accessible through the mapped port.

Step 1: Pull the Ubuntu Image

We download the Ubuntu image from Docker Hub so that we can create a container from it.

Command

`docker pull ubuntu`

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
6f5c5aa4e145: Already exists
1c24335ddd46: Already exists
Digest: sha256:f3d28607ddd78734bb7f71f117f3c6706c666b8b76cbff7c9ff6e5718d46ff64
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
```

Explanation of attributes

- **docker pull** → Downloads an image from Docker Hub.
- **ubuntu** → Name of the image to download.

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
ubuntu        latest    30ba44506a6d   6 weeks ago   100MB
server        latest    14bac6b71a17   2 years ago   912MB
postgres      16.1-alpine3.18 b467784150b4   2 years ago   239MB
```

Step 2: Create and Run the Container with Port Mapping

We create a container named **web_container** and map host port **8080** to container port **80** so that web traffic from the host can reach the Apache server inside the container.

`docker run -it -d --name web_container -p 8080:80 ubuntu bash`

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker run -it --name web_container -p 8080:80 ubuntu
dd0c0d66228ce854a97f3252f82d159949877a13141bf18879616eb85b07c054
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                    NAMES
dd0c0d66228c   ubuntu   "/bin/bash"             4 seconds ago Up 3 seconds   0.0.0.0:8080->80/tcp    web_container
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> d
```

Explanation of attributes

- `docker run` → Creates and starts a container.
- `-i` → Keeps STDIN open.
- `-t` → Allocates a terminal.
- `-d` → Runs container in background.
- `--name web_container` → Assigns container name.
- `-p 8080:80` → Maps host port 8080 to container port 80.
- `ubuntu` → Source image.
- `bash` → Starts bash shell.

Step 3: Verify Container Status

We confirm that the container is running properly before configuring Apache.

Command

`docker ps`

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker run -dit --name web_container -p 8080:80 ubuntu
dd0c0d66228ce854a97f3252f82d159949877a13141bf18879616eb85b07c054
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
dd0c0d66228c   ubuntu   "/bin/bash"             4 seconds ago Up 3 seconds  0.0.0.0:8080->80/tcp               web_container
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> d
```

Explanation of attributes

- `docker ps` → Displays running containers.

Step 4: Access the Running Container

We enter the container to install and configure Apache.

`docker exec -it web_container bash`

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker exec -it web_container bash
root@dd0c0d66228c:/# apt update && apt install apache2 -y
Get:1 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:2 http://archive.ubuntu.com/ubuntu resolute InRelease [136 kB]
Get:3 http://security.ubuntu.com/ubuntu resolute-security/restricted amd64 Packages [250 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [236 kB]
Get:5 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [114 kB]
Get:6 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:7 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [136 kB]
Get:8 http://archive.ubuntu.com/ubuntu resolute/universe amd64 Packages [20.1 MB]
Get:9 http://archive.ubuntu.com/ubuntu resolute/main amd64 Packages [1874 kB]
Get:10 http://archive.ubuntu.com/ubuntu resolute/multiverse amd64 Packages [352 kB]
Get:11 http://archive.ubuntu.com/ubuntu resolute/restricted amd64 Packages [189 kB]
Get:12 http://archive.ubuntu.com/ubuntu resolute-updates/multiverse amd64 Packages [3584 B]
Get:13 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [244 kB]
Get:14 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [120 kB]
Get:15 http://archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Packages [250 kB]
Fetched 24.3 MB in 5s (4754 kB/s)
3 packages can be upgraded. Run 'apt list --upgradable' to see them.
Installing:
apache2
```

Explanation of attributes

- `docker exec` → Executes command inside container.
- `-it` → Interactive terminal.
- `web_container` → Target container.
- `bash` → Opens shell.

Step 5: Update Package Repository

We update package information before installing Apache.

```
apt-get update
```

Explanation of attributes

- `apt-get update` → Downloads latest package information.

Step 6: Install Apache2

We install the Apache web server to host a web page inside the container.

Command

```
apt-get install -y apache2
```

Explanation of attributes

- `apt-get install` → Installs package.
- `-y` → Automatically answers Yes.
- `apache2` → Apache web server package.

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker exec -it web_container bash
root@dd0c0d66228c:/# apt update && apt install apache2 -y
Get:1 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:2 http://archive.ubuntu.com/ubuntu resolute InRelease [136 kB]
Get:3 http://security.ubuntu.com/ubuntu resolute-security/restricted amd64 Packages [250 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [236 kB]
Get:5 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [114 kB]
Get:6 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:7 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [136 kB]
Get:8 http://archive.ubuntu.com/ubuntu resolute/universe amd64 Packages [20.1 MB]
Get:9 http://archive.ubuntu.com/ubuntu resolute/main amd64 Packages [1874 kB]
Get:10 http://archive.ubuntu.com/ubuntu resolute/multiverse amd64 Packages [352 kB]
Get:11 http://archive.ubuntu.com/ubuntu resolute/restricted amd64 Packages [189 kB]
Get:12 http://archive.ubuntu.com/ubuntu resolute-updates/multiverse amd64 Packages [3584 B]
Get:13 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [244 kB]
Get:14 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [120 kB]
Get:15 http://archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Packages [250 kB]
Fetched 24.3 MB in 5s (4754 kB/s)
3 packages can be upgraded. Run 'apt list --upgradable' to see them.
Installing:
apache2
```

Step 7: Start Apache Service

We start the Apache web server so it can serve web pages.

Command

```
service apache2 start
```

Explanation of attributes

- **service** → Manages Linux services.
- **apache2** → Apache service.
- **start** → Starts the service.

```
root@dd0c0d66228c:/# service apache2 start
* Starting Apache httpd web server apache2
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directive globally to suppress this message
*
root@dd0c0d66228c:/# service apache2 start
* Starting Apache httpd web server apache2
*
root@dd0c0d66228c:/# systemctl status httpd
bash: systemctl: command not found
root@dd0c0d66228c:/# apt install systemctl -y
Installing:
  systemctl

Installing dependencies:
  libpython3-stdlib  libpython3.14-stdlib  python3  python3.14  readline-common
  libpython3.14-minimal  libreadline8t64  python3-minimal  python3.14-minimal  tzdata

Suggested packages:
  python3-doc  python3-tk  python3-venv  python3.14-venv  python3.14-doc  binutils  binfmt-support  readline-doc  tini | dumb-init
```

Step 8: Create a Custom Web Page

We create a custom HTML page displaying the PRN number.

Command

```
echo "<h1>PRN - 260240123032</h1>" > /var/www/html/index.html
```

```
root@dd0c0d66228c:/# echo "<h1>PRN - 260240123060 </h1>" > /var/www/html/index.html
root@dd0c0d66228c:/# cat /var/www/html/index.html
<h1>PRN - 260240123060 </h1>
```

Explanation of attributes

- **echo** → Writes content.
- **>** → Creates/overwrites file.
- **/var/www/html/index.html** → Default Apache web page.

Step 9: Verify Web Page Inside Container

We confirm that the web page contains the correct content.

Command

```
cat /var/www/html/index.html
```

```
root@dd0c0d66228c:/# echo "<h1>PRN - 260240123060 </h1>" > /var/www/html/index.html
root@dd0c0d66228c:/# cat /var/www/html/index.html
<h1>PRN - 260240123060 </h1>
```

Explanation of attributes

- **cat** → Displays file contents.

```
root@dd0c0d66228c:/# apt install curl -y
Installing:
  curl

Installing dependencies:
  bash-completion

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 3
  Download size: 486 kB
  Space needed: 2520 kB / 1017 GB available

Get:1 http://archive.ubuntu.com/ubuntu resolute/main amd64 bash-completion all 1:2.16.0-8build1 [214 kB]
Get:2 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 curl amd64 8.18.0-1ubuntu2.1 [272 kB]
Fetched 486 kB in 11s (43.2 kB/s)
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 79, <STDIN> line 2.)
debconf: falling back to frontend: Readline
Selecting previously unselected package bash-completion.
(Reading database ... 12896 files and directories currently installed.)
Preparing to unpack .../bash-completion_1%3a2.16.0-8build1_all.deb ...
Unpacking bash-completion (1:2.16.0-8build1) ...
Selecting previously unselected package curl.
Preparing to unpack .../curl_8.18.0-1ubuntu2.1_amd64.deb ...
Unpacking curl (8.18.0-1ubuntu2.1) ...
Setting up bash-completion (1:2.16.0-8build1) ...
Setting up curl (8.18.0-1ubuntu2.1) ...
root@dd0c0d66228c:/# curl 127.0.0.1
<h1>PRN - 260240123060 </h1>
root@dd0c0d66228c:/#
```

Step 10: Exit the Container

We leave the container after configuration is complete.

Command

```
exit
```

Explanation of attributes

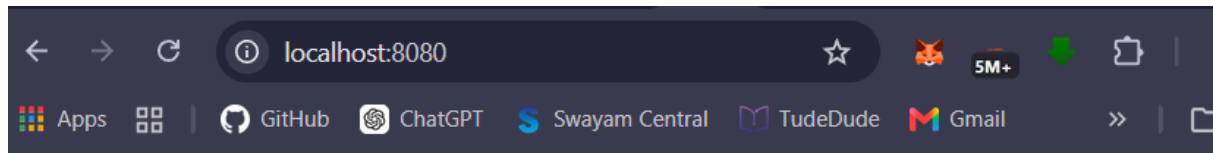
- **exit** → Closes container shell.

Step 11: Access the Web Page from Host Machine

We verify that port mapping works correctly.

URL

`http://localhost:8080`



PRN - 260240123060

Step 12: Access the Web Page from Another Machine

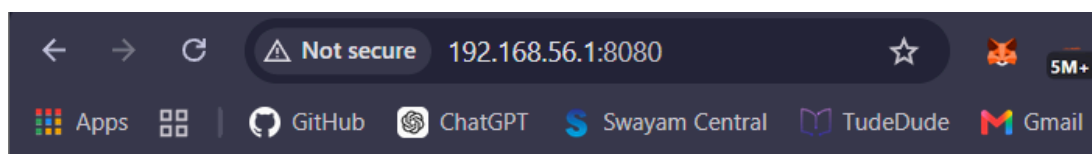
We verify that the web page is accessible remotely through the host's IP address.

URL

`http://<HOST_IP>:8080`

Explanation

- `<HOST_IP>` → IP address of Docker host machine.
- `8080` → Host port mapped to Apache port 80.



PRN - 260240123060

Memory Sheet (Command Flow)

`docker pull ubuntu`

`docker run -it -d --name web_container -p 8080:80 ubuntu bash`

`docker ps`

`docker exec -it web_container bash`

`apt-get update`

`apt-get install -y apache2`

`service apache2 start`

`echo "<h1>PRN - 260240123032</h1>" > /var/www/html/index.html`

`cat /var/www/html/index.html`

`exit`

`# Browser`

`http://localhost:8080`

`# Another machine`

`http://<HOST_IP>:8080`

Q3) Docker Environment Variables (-e)

Create a container named **env_container** using the Nginx image and pass environment variables for **NAME**, **PRN**, and **COURSE** during container creation. Access the container and verify that the environment variables have been set correctly. Modify the web page to display the values stored in the environment variables and verify the output through a browser.

Step 1: Pull the Nginx Image

We download the Nginx image from Docker Hub so that we can create a web server container.

Command

docker pull nginx

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
5b4d6ff92fc4: Pull complete
830625e1ac85: Pull complete
5431d0092ffd: Pull complete
b4a248c845e5: Pull complete
7f8b1a2b17d8: Pull complete
45381ecb0e2f: Pull complete
13fd728be9eb: Pull complete
Digest: sha256:5aca99593157f4ae539a5dec1092a0ad8762f8e2eb1789085a13a0f5622369f6
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest

What's next:
  View a summary of image vulnerabilities and recommendations → docker scout quickview nginx
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nginx	latest	7aaca76c508f	2 weeks ago	161MB
ubuntu	latest	30ba44506a6d	6 weeks ago	100MB
server	latest	14bac6b71a17	2 years ago	912MB
postgres	16.1-alpine3.18	b467784150b4	2 years ago	239MB

Explanation of attributes

- **docker pull** → Downloads an image from Docker Hub.
- **nginx** → Official Nginx web server image.

Step 2: Create Container with Environment Variables

We create a container named **env_container** and pass environment variables during container creation.

Command

```
docker run -d --name env_container -p 8080:80 \
-e NAME="John J. Johnson Jr." \
-e PRN="260240123060" \
-e COURSE="DITISS" \
nginx
```

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker run -dit --name env_container -p 8080:80 -e NAME="John J. Johnson Jr." -e PRN="260240123060" -e branch="DITISS" nginx bash
d1453055122492cedbbc600b016874ec42f4d8b5843d81a0cc8291c4b5e1bb16
```

Explanation of attributes

- **docker run** → Creates and starts a container.
- **-d** → Runs container in background.
- **--name env_container** → Assigns container name.
- **-p 8080:80** → Maps host port 8080 to container port 80.
- **-e** → Sets environment variables.
- **NAME** → Student name variable.
- **PRN** → PRN number variable.
- **COURSE** → Course name variable.
- **nginx** → Source image.

Step 3: Verify Container is Running

We ensure the container has started successfully.

Command

```
docker ps
```

Explanation of attributes

- **docker ps** → Displays running containers.

Step 4: Access the Container

We enter the running container to inspect the environment variables.

Command

```
docker exec -it env_container bash
```

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker exec -it env_container bash
root@d14530551224:/# echo $NAME && echo $PRN && echo $branch
John J. Johnson Jr.
260240123060
DITISS
root@d14530551224:/# |
```

Explanation of attributes

- `docker exec` → Executes commands inside container.
- `-it` → Interactive terminal.
- `env_container` → Target container.
- `bash` → Opens shell.

Step 5: Verify Environment Variables

We confirm that the variables passed during container creation are available inside the container.

Command

```
echo $NAME
```

```
echo $PRN
```

```
echo $COURSE
```

```
PS D:\learn\itim\itim\docker\newLab\q1.2_dockerHub\studentapp> docker exec -it env_container bash
root@d14530551224:/# echo $NAME && echo $PRN && echo $branch
John J. Johnson Jr.
260240123060
DITISS
root@d14530551224:/# |
```

Explanation of attributes

- `echo` → Displays variable value.
- `$NAME` → Student name.
- `$PRN` → PRN number.
- `$COURSE` → Course name.

Step 6: Create a Custom Web Page

We modify the Nginx web page to display the environment variable values.

Command

```
cat > /usr/share/nginx/html/index.html
```

File Content

```
<html>
<head>
<title>Docker Environment Variables</title>
</head>
<body>
<h1>Student Details</h1>
<p>Name: John J. Johnson Jr.</p>
<p>PRN: 260240123060</p>
<p>Course: DITISS</p>
</body>
</html>
```

Press:

CTRL + D

to save.

Explanation of attributes

- `/usr/share/nginx/html/index.html` → Default Nginx web page.
- `cat >` → Creates a new file.

Step 7: Verify Web Page Inside Container

We ensure that the web page contains the expected content.

Command

```
cat /usr/share/nginx/html/index.html
```

```
root@d14530551224:/# cat > /usr/share/nginx/html/index.html
<html>
<head>
<title>Docker Environment Variables</title>
</head>
<body>
<h1>Student Details</h1>
<p>Name: John J. Johnson Jr.</p>
<p>PRN: 260240123060</p>
<p>Course: DITISS</p>
</body>
</html>
root@d14530551224:/# cat /usr/share/nginx/html/index.html
<html>
<head>
<title>Docker Environment Variables</title>
</head>
<body>
<h1>Student Details</h1>
<p>Name: John J. Johnson Jr.</p>
<p>PRN: 260240123060</p>
<p>Course: DITISS</p>
</body>
</html>
root@d14530551224:/# |
```

Step 8: Exit the Container

We leave the container after configuration is complete.

Command

exit

Explanation of attributes

- `exit` → Closes container shell.

Step 9: Verify Output Through Browser

We verify that the web page is accessible through the browser.

URL

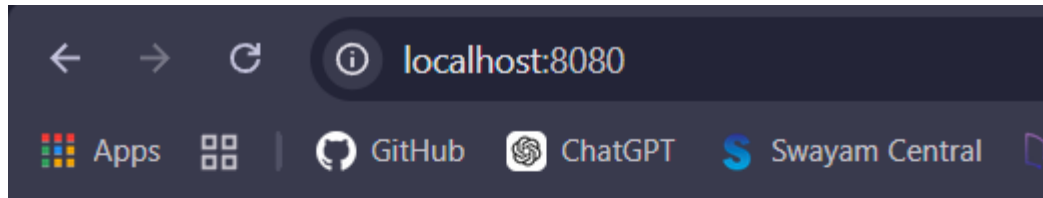
<http://localhost:8080>

Step 10: Verify from Another Machine

We confirm that the page is accessible remotely through the Docker host IP.

URL

`http://<HOST_IP>:8080`



Student Details

Name: \$NAME

PRN: \$PRN

Course: \$COURSE

Explanation of attributes

- `<HOST_IP>` → IP address of Docker host machine.
- `8080` → Host port mapped to Nginx port 80.

Memory Sheet (Command Flow)

```
docker pull nginx
```

```
docker run -d --name env_container -p 8080:80 \  
-e NAME="John J. Johnson Jr." \  
-e PRN="260240123060" \  
-e COURSE="DITISS" \  
nginx
```

```
docker ps
```

```
docker exec -it env_container bash
```

```
echo $NAME  
echo $PRN  
echo $COURSE
```

```
cat > /usr/share/nginx/html/index.html
```

```
cat /usr/share/nginx/html/index.html
```

```
exit
```

```
# Browser  
http://localhost:8080
```

```
# Another Machine  
http://<HOST_IP>:8080
```

Q4) Docker Named Volume

Create a **named volume** called **student_data**. Launch a container named **vol_container** and attach the volume to a directory inside the container. Create a file containing your name and PRN number in the mounted directory. Remove the container and create a new container using the same volume. Verify that the previously created file is still available.

```
—
# Create named volume to store persistent data
docker volume create student_data

# Create container and attach named volume
docker run -dit --name vol_container -v student_data:/data ubuntu
bash

# Create file inside volume
docker exec -it vol_container bash

echo "Name: Adi, PRN: 260240123060" > /data/student.txt

cat /data/student.txt

exit

# Remove container (volume remains)
docker rm -f vol_container

# Create new container using same volume
docker run -dit --name vol_container2 -v student_data:/data ubuntu
bash

# Verify data persists
docker exec -it vol_container2 cat /data/student.txt
```

Q5) Docker Anonymous Volume

Create a container using an **anonymous volume** mounted to a data directory. Create a file containing your PRN number and verify its existence. Remove the container and create a new container from the same image. Verify that the previously stored data is not automatically available and explain the behavior of anonymous volumes.

```
# Create container with anonymous volume
docker run -dit --name anon_container -v /data ubuntu bash

# Create file inside anonymous volume
docker exec -it anon_container bash

echo "PRN: 260240123060" > /data/prn.txt

cat /data/prn.txt

exit

# Remove container
docker rm -f anon_container

# Create new container from same image
docker run -dit --name anon_container2 -v /data ubuntu bash

# Verify old data is not available
docker exec -it anon_container2 ls /data
```

Q6) Docker Bind Mount

Create a directory on the host machine named **docker_bind_data** and place a file containing your name and PRN number inside it. Launch a container named **bind_container** and bind mount the host directory into the container. Verify that the file is accessible from inside the container. Modify the file from within the container and verify that the changes are reflected on the host machine. Finally, modify the file on the host and verify that the updated content is visible inside the container.

Windows Host

Create host directory

```
mkdir docker_bind_data
```

Create file on host

```
echo "Name: Adi PRN:260240123060" > docker_bind_data\student.txt
```

Run Container

Bind host folder inside container

```
docker run -dit --name bind_container -v  
${PWD}/docker_bind_data:/data ubuntu bash
```

Verify Host File Inside Container

Access container

```
docker exec -it bind_container bash
```

View file

```
cat /data/student.txt
```

Modify File From Container

```
# Update file from container
```

```
echo "Modified From Container" >> /data/student.txt
```

```
cat /data/student.txt
```

```
exit
```

Verify On Host

```
# Changes visible on host
```

```
type docker_bind_data\student.txt
```

Modify From Host

```
# Update from host
```

```
echo Updated_From_Host >> docker_bind_data\student.txt
```

Verify Inside Container

```
# Check updated content
```

```
docker exec -it bind_container cat /data/student.txt
```

Result

Host ↔ Container share same file.

Changes on either side are immediately reflected.